

Objetos y tipos de datos en Python

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction	2
1 Listas	2
2 Asignación como referencia o como valor	2
3 Evaluando la existencia de objetos en una lista	3
4 Otros metodos para modificar listas.	3
5 Trabajando con tuplas.	4
6 Diccionarios:	5
7 Publicaciones Similares	5

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

Objetos y tipos de datos en Python

1 Listas

Las listas son objetos mutables e iterables en python. Son muy versátiles pues permiten coleccionar de manera ordenada cualquier objeto (incluso otras listas). Las listas además te permiten acceder a los elementos que contiene adentro utilizando índices.

```
frutas = ['manzana', 'banana', 'durazno']
print(frutas)
```

```
carnes = list(['cerdo', 'vaca', 'pollo'])
print(carnes)
```

```
mix = [123, ['Python', 'Stata'], 'iPhone']
print(mix)
```

2 Asignación como referencia o como valor

Muchos cometen el error de ‘copiar’ una lista de manera directa y luego editarla. El resultado es que todas las ediciones que hagamos en la lista referida se replicaran en la lista original. Esto ocurre cuando asignamos una lista como referencia en lugar de copiarla.

Cuando la asignamos como referencia lo unico que hace python es usar el nuevo nombre de variable para referenciar la variable original. Esto permite a python salvar memoria ram porque no estamos creando más objetos en el ambiente de python.

```
import copy

miPrimeraLista = [1, 2, 3, 4, 5]
miPrimeraLista1 = miPrimeraLista # Esto asigna como referencia
miPrimeraLista2 = copy.copy(miPrimeraLista) # Esto asigna como valor
#miPrimeraLista[0]
miPrimeraLista[0] = 3
miPrimeraLista[1] = 9

print('miPrimeraLista:', miPrimeraLista)
print('miPrimeraLista1:', miPrimeraLista1)
print('miPrimeraLista2:', miPrimeraLista2)
```

En el caso de listas de listas la función que nos permite copiar el objeto completo es la rutina deepcopy. Basicamente lo que hace deepcopy es copiar como valor no solo la lista inicial, sino tambien las listas que se contienen adentro.

```
#List of lists:
miSegundaLista = [{"Jose", "Pablo"}, [26, 25]]
miSegundaListaNombres = miSegundaLista # Esto asigna como referencia
miSegundaListaNombres2 = copy.deepcopy(miSegundaLista) # Deepcopy asigna como valor
miSegundaLista[1][1] = 24
```

```
print('Lista original:',miSegundaLista)
print('Lista copiada como referencia:',miSegundaListaNombres)
print('Lista copiada como valor:',miSegundaListaNombres2)
```

Para eliminar elementos dentro de una lista, podemos indexar el objeto y con el comando `del`, podemos eliminar dicho objeto.

```
del miSegundaLista[1]
miSegundaLista
```

```
# Combinar listas
mylist = [1, 2] + [3, 4]
print(mylist)
```

```
[1, 2]*2
```

3 Evaluando la existencia de objetos en una lista

En python es posible evaluar la membresía de un elemento en colecciones de elementos como listas, sets, diccionarios y tuplas. Para ello existen los comandos “in”, “not in”.

Así mismo es posible evaluar si un objeto en python es idéntico a otro. Esta rutina evalúa si un objeto tiene el mismo valor y propiedades que otro, para ello se usa la rutina “is”.

```
# Evaluar si un objeto se encuentra en una lista
print(mylist)
print(1 in mylist)
print(5 in mylist)
print(1 not in mylist)
print(5 not in mylist)
```

```
print(5 is 5)
print(5 is 5.)
```

4 Otros metodos para modificar listas.

Existen además otros métodos que nos permiten modificar las listas. Por ejemplo, podemos obtener el valor máximo, el mínimo, el tamaño de la lista, así como también podemos agregar y remover elementos a una lista.

```
# Metodos en listas
print(len(mylist)) #Tamano de lista
print(min(mylist)) #Valor minimo en la lista
print(max(mylist)) #Valor maximo en la lista
```

```
# Metodos para agregar y eliminar elementos de una lista:
mylist.append(5) #Agregar elemento
mylist
```

```
mylist.append([6,7])  
mylist
```

```
mylist.remove([6,7]) # Remover elemento  
mylist
```

```
mylist.extend([6,7]) # Extender lista con mas elementos  
mylist
```

```
mylist.insert(1,10) # insert element to a list at a location other than the end  
mylist
```

```
mylist.sort()  
print(mylist) # Ordenar lista de menor a mayor  
mylist.index(2) # Retorna el indice del valor que solicitamos
```

```
mylistExample = [1,2,2,4,2]  
mylistExample.index(2) # Retorna el indice del valor que solicitamos
```

5 Trabajando con tuplas.

Las tuplas son colecciones ordenadas de elementos que son inmutables. El beneficio de que sea inmutable es que definir una tupla requiere poca memoria ram pues no tiene muchas rutinas asociadas a comparación de las listas. Usualmente definimos las tuplas entre paréntesis “(,)”.

```
import sys  
  
# Uso de ram para tuplas vs listas:  
myTuple = tuple([i for i in range(1000)])  
myList = [i for i in range(1000)]  
print(sys.getsizeof(myTuple),  
      sys.getsizeof(myList))
```

```
#Evaluando el tipo de elemento  
myTuple = 1,2,3  
#print(myTuple)  
type(myTuple)
```

```
# Convirtiendo una lista en tupla  
tuple([1,2])
```

```
# Indexando una tupla  
myTuple[0]
```

6 Diccionarios:

Los diccionarios son colecciones de elementos de manera no ordenada, son mutables y posibles de indexar. En python la forma de crear diccionarios es usando llaves “{}”. Los diccionarios se utilizan para elaborar estructuras de datos que contienen multiples objetos y usualmente se pueden referenciar on un “key name” que suele ser un string. Por lo general los diccionarios siguen la siguiente estructura:

```
dictorinary = { 'key1': obj1,
                'key2': obj2,
                'key3': obj3}
```

```
# Diccionarios
miDict = {}

miDict = {'a':'perro', 'b':'gato'}
print(miDict['a'])
```

```
miDict['a'] = 'canario'
miDict['c'] = 'conejo'

print(miDict)
```

```
#Acceder a llaves (keys) en los diccionarios:
print('a' in miDict,
      'canario' in miDict)
```

```
list(miDict)
```

```
miDict.keys()
```

```
miDict.values()
```

```
miDict.items()
```

7 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Instalacion De Anaconda](#)
2.  [Configurar Entorno Virtual Python Anaconda](#)
3.  [01 Introducion A La Programacion Con Python](#)
4.  [02 Variables Expresiones Y Statements Con Python](#)
5.  [03 Objetos De Python](#)
6.  [04 Ejecucion Condicional Con Python](#)
7.  [05 Iteraciones Con Python](#)

8.  06 Funciones Con Python
9.  07 Dataframes Con Python
10.  08 Prediccion Y Metrica De Performance Con Python
11.  09 Metodos De Machine Learning Para Clasificacion Con Python
12.  10 Metodos De Machine Learning Para Regresion Con Python
13.  11 Validacion Cruzada Y Composicion Del Modelo Con Python
14.  Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!